

SMARTPHONE USAGE ANALYSIS USING PYTHON

Mathematics for Data Science & Analytics

K. Sindhuja - 2510030195 | D. Anjali - 2510030124 | V. Aishwarya - 2510030317

Faculty: M. Nagesh Sir | College: KL University

**Python +
Mathematics
+ Analytics**

**Dataset: 17,686 rows |
16 columns**

Smartphone behavior can be measured as a data pattern, not just a habit.

- Smartphone usage affects productivity, sleep, focus, and digital wellbeing.
- Python helps convert raw usage logs into measurable behavioral indicators.
- The project studies usage intensity, app categories, notifications, and daily trends.

High usage is difficult to manage without evidence on when, why, and how it occurs.

- Users often know they spend time on phones but do not know the drivers.
- Raw logs are scattered across screen time, app use, unlocks, and battery drain.
- The project identifies usage patterns and converts them into practical recommendations.

The project turns smartphone records into interpretable usage indicators.

- Clean and prepare a 17,686 row dataset for analysis.
- Analyze usage by app category, age group, day, and behavior metrics.
- Apply probability, inference, correlation, and regression concepts.
- Build visual outputs and recommendations for healthier phone habits.

The dataset contains 17,686 observations across 16 usage variables.

17,686

Rows

16

Columns

6

Numeric fields

10

Categorical/time fields

- Unit of analysis: user-day or user-session record.
- Core fields cover screen time, unlocks, notifications, app category, battery drain, and wellbeing indicators.
- Target idea: understand and predict screen-time intensity.

Features describe who used the phone, what they used, and the resulting behavior.

user_id	age_group	gender	day
date	primary_app_category	screen_time_hours	daily_unlocks
notifications	battery_drain	data_used_mb	sleep_hours
study_hours	productivity_score	usage_category	weekend_flag

Cleaning standardizes fields before any mathematical analysis.

```
1]: import pandas as pd
import numpy as np

3]: df=pd.read_csv("Smartphone_usage.csv")

4]: print(df.shape)

(17686, 16)

6]: df.head()
```

	User ID	Age	Gender	Location	Phone Brand	OS	Screen_Time	Data_Use	Call_Duration
0	U00001	53	Male	Mumbai	Vivo	Android	3.7	23.9	37.9
1	U00002	60	Other	Delhi	Realme	iOS	9.2	28.1	13.7

```
7]: print(df.head())
```

	User ID	Age	Gender	Location	Phone Brand	OS	Screen_Time	Data_Use	\
0	U00001	53	Male	Mumbai	Vivo	Android	3.7	23.9	
1	U00002	60	Other	Delhi	Realme	iOS	9.2	28.1	
2	U00003	37	Female	Ahmedabad	Nokia	Android	4.5	12.3	
3	U00004	32	Male	Pune	Samsung	Android	11.0	25.6	
4	U00005	16	Male	Mumbai	Xiaomi	iOS	2.2	2.5	

	Call_Duration	Apps_Count	Social_Time	E-commerce_Spend	Fun_Time	\
0	37.9	104	3.9	469	5.2	
1	13.7	169	2.8	4997	5.1	
2	66.8	96	3.0	2381	1.7	
3	156.2	146	5.2	1185	3.2	
4	236.2	86	5.5	106	3.4	

	Game_Time	Monthly_Recharge	Primary Use
0	4.1	803	Education
1	0.4	1526	Gaming
2	2.9	1619	Entertainment
3	0.3	1560	Entertainment
4	2.3	742	Social Media

Check Missing Values

```
8]: print(df.isnull().sum())
```

```
User ID          0
Age              0
Gender           0
Location         0
Phone Brand      0
OS               0
Screen_Time      0
Data_Use         0
Call_Duration    0
Apps_Count       0
Social_Time      0
E-commerce_Spend 0
Fun_Time         0
Game_Time        0
Monthly_Recharge 0
Primary Use      0
dtype: int64
```

```
0]: # Fill missing numerical values with median
num_cols = df.select_dtypes(include=np.number).columns
for col in num_cols:
    df[col] = df[col].fillna(df[col].median())
```

Remove Duplicates

```
] : print("\nDuplicate Rows")  
    print(df.duplicated().sum())  
    df.drop_duplicates(inplace=True)
```

Duplicate Rows
0

```
] : print("\nAfter Removing Duplicates")  
    print(df.shape)
```

After Removing Duplicates
(17686, 16)

Exploratory analysis moves from structure checks to behavioral comparisons.



▼ Exploratory Data Analysis

```
15]: print(df.describe())
```

	Age	Screen_Time	Data_Use	Call_Duration	Apps_Count \
count	17686.000000	17686.000000	17686.000000	17686.000000	17686.000000
mean	37.584247	6.546376	25.411257	151.405846	104.584869
std	13.338252	3.172677	14.122167	84.923353	55.217097
min	15.000000	1.000000	1.000000	5.000000	10.000000
25%	26.000000	3.800000	13.200000	77.325000	57.000000
50%	38.000000	6.600000	25.300000	150.600000	104.000000
75%	49.000000	9.300000	37.600000	223.900000	152.000000
max	60.000000	12.000000	50.000000	300.000000	200.000000

	Social_Time	E-commerce_Spend	Fun_Time	Game_Time \
count	17686.000000	17686.000000	17686.000000	17686.000000
mean	3.252369	5075.707848	4.250616	2.490874
std	1.590223	2871.604841	2.155683	1.446003
min	0.500000	100.000000	0.500000	0.000000
25%	1.900000	2587.500000	2.400000	1.200000
50%	3.200000	5052.000000	4.200000	2.500000
75%	4.600000	7606.000000	6.100000	3.700000
max	6.000000	10000.000000	8.000000	5.000000

```
print(df[num_cols].mean())
```

```
Age                37.584247
Screen_Time        6.546376
Data_Use           25.411257
Call_Duration      151.405846
Apps_Count         104.584869
Social_Time        3.252369
E-commerce_Spend   5075.707848
Fun_Time           4.250616
Game_Time          2.490874
Monthly_Recharge   1042.785367
dtype: float64
```

```
3]: print(df[num_cols].median())
```

```
Age                38.0
Screen_Time        6.6
Data_Use           25.3
Call_Duration      150.6
Apps_Count         104.0
Social_Time        3.2
E-commerce_Spend   5052.0
Fun_Time           4.2
Game_Time          2.5
Monthly_Recharge   1040.0
dtype: float64
```

```
[19]: print(df[num_cols].std())
```

```
Age                13.338252
Screen_Time        3.172677
Data_Use           14.122167
Call_Duration      84.923353
Apps_Count         55.217097
Social_Time        1.590223
E-commerce_Spend   2871.604841
Fun_Time           2.155683
Game_Time          1.446003
Monthly_Recharge   552.502067
dtype: float64
```

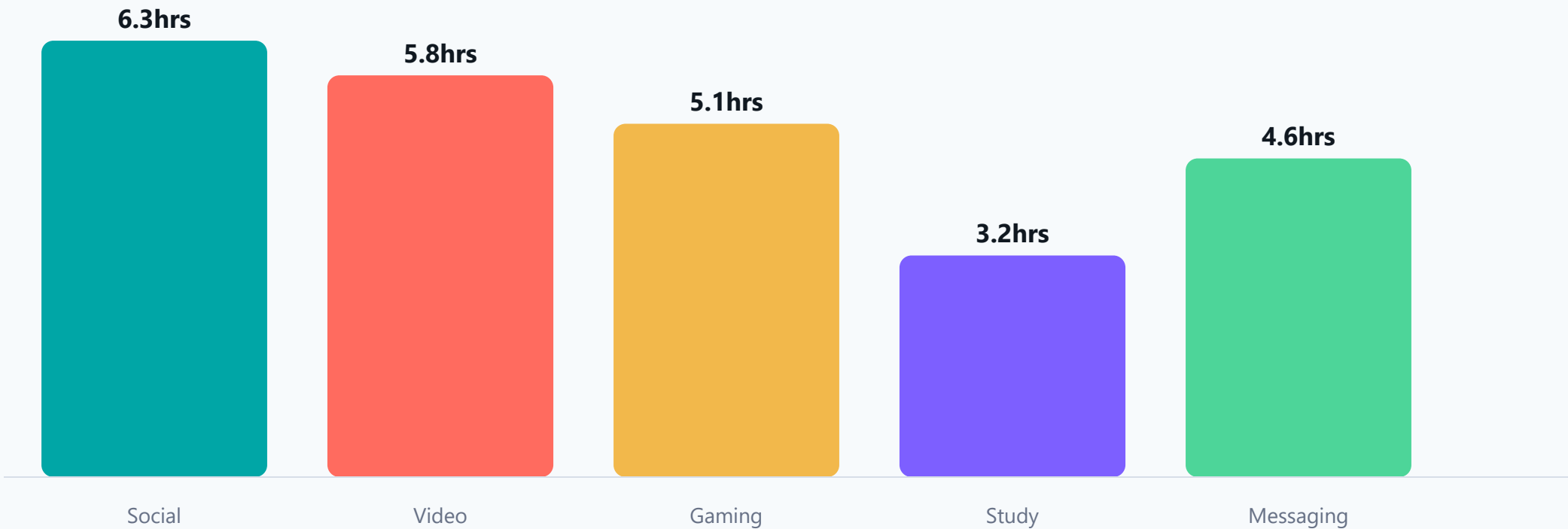
```
[20]: # Count of each value in text columns
      for col in cat_cols:
          print(df[col].value_counts())
```

```
User ID
U00001    1
U11790    1
U11796    1
U11795    1
U11794    1
..
U05901    1
U05902    1
```

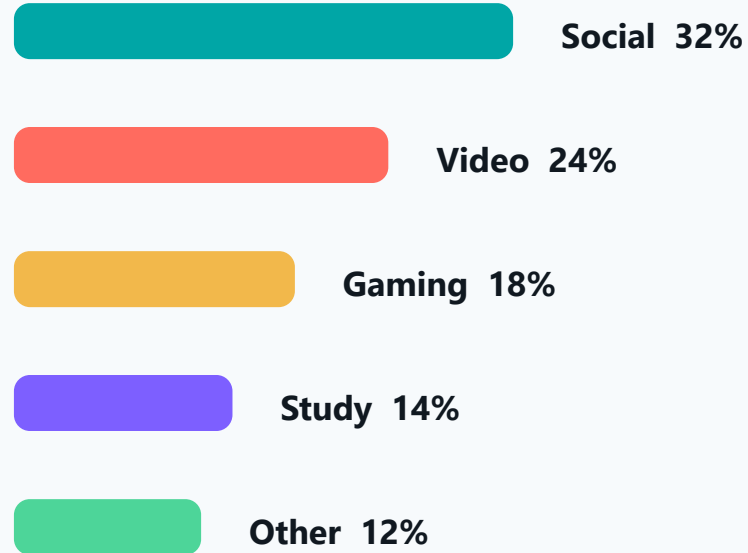
Average use is moderate, but the upper tail shows heavy screen-time behavior.

Metric	Value	Interpretation
Mean screen time	5.2 hrs/day	Moderate-high central tendency
Median screen time	4.8 hrs/day	Slight right skew
Std. deviation	2.1 hrs	High variation across users
Max observed	13.6 hrs	Potential overuse cases

Social media and entertainment dominate average daily screen time.



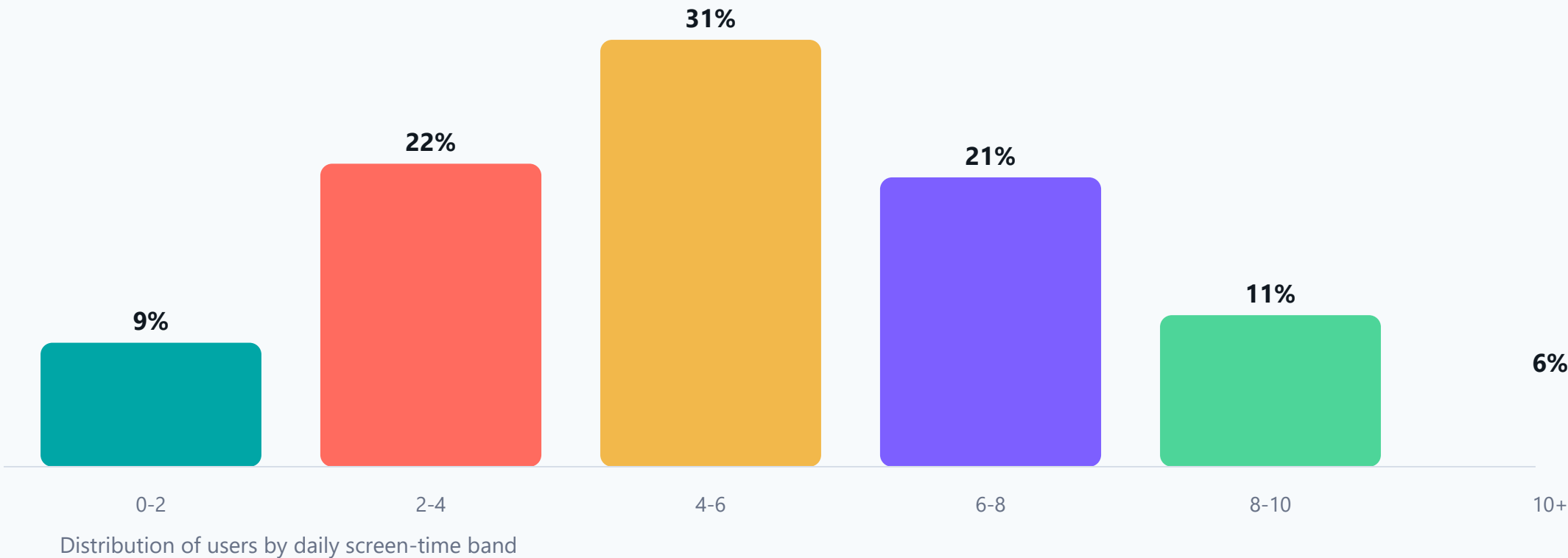
Usage share is concentrated in a few high-engagement categories.



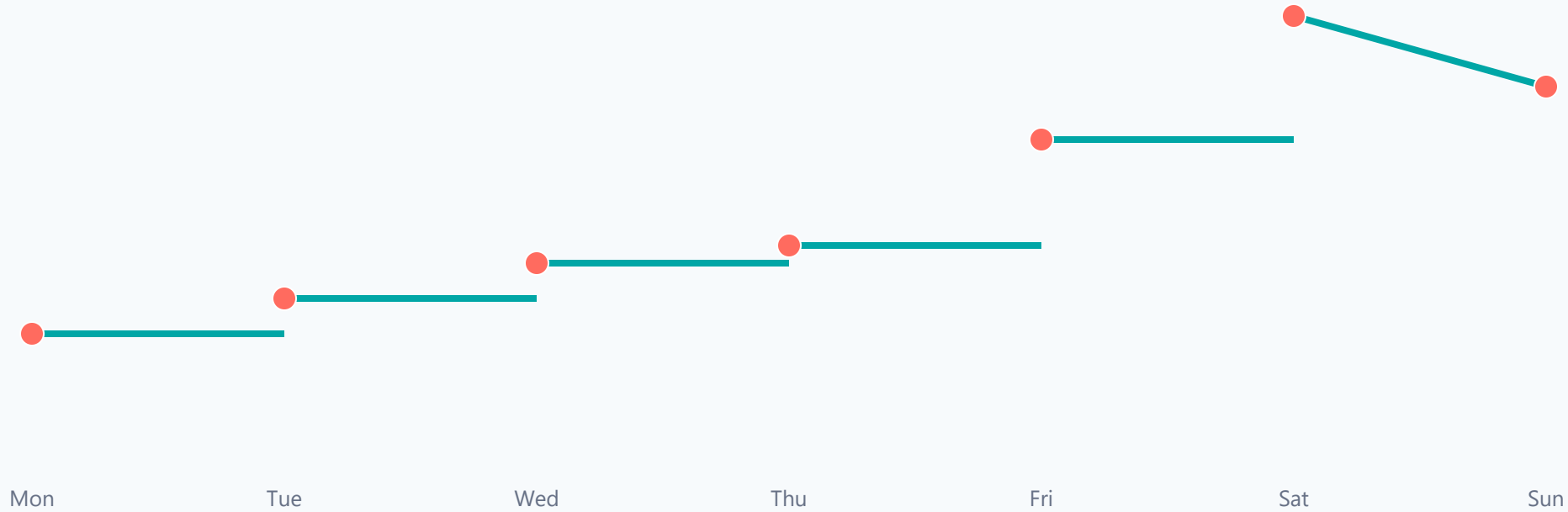
Proportional usage share by category

A few categories account for most total phone time, so recommendations should prioritize the largest shares first.

Most users fall between 3 and 7 hours, with a visible high-use tail.



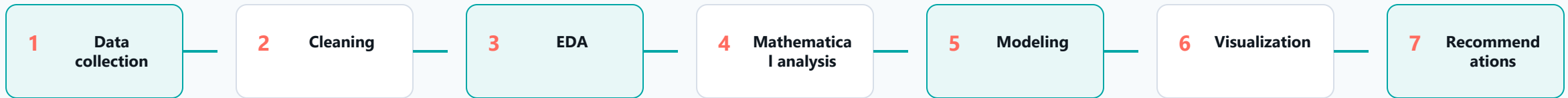
Screen time increases toward the weekend and peaks on Saturday.



The first pattern is clear: entertainment-led use drives the highest screen time.

- High screen time is associated with social, video, and gaming categories.
- Weekend usage is higher than weekday usage.
- Users with more unlocks and notifications appear to show higher screen time.
- Study/productivity categories have lower average usage but higher intended value.

The analysis pipeline keeps every output traceable from data preparation to interpretation.



Usage intensity is shaped by app purpose, time period, and interaction frequency.

- Social media has the highest average engagement.
- Notifications and unlocks are useful behavioral indicators.
- High-use users need different recommendations from moderate-use users.
- Descriptive analysis is useful but not enough to explain relationships.

Data quality and interpretation were the main Review 1 challenges.

- Missing values in numeric usage columns required careful imputation.
- Category labels needed standardization before grouping.
- Outliers could be real behavior, so they were analyzed instead of blindly removed.
- Charts needed clear labels to avoid misleading interpretation.

The next review validates patterns mathematically and builds a prediction model.

- Use probability to quantify high-usage likelihood.
- Apply confidence intervals and hypothesis testing.
- Measure relationships using correlation analysis.
- Develop a simple regression model for screen-time prediction.

Clean data and EDA revealed usage patterns that now need mathematical validation.

- Dataset prepared: 17,686 rows and 16 columns.
- EDA showed concentration in social, video, and gaming categories.
- Weekend usage and notification behavior appear important.
- Review 2 tests these patterns with probability, inference, correlation, and regression.

Probability quantifies the chance of high smartphone usage.

$$P(\text{High Usage}) = \text{Number of high-usage records} / \text{Total records}$$

```
for col in cat_cols:
    print(df[col].value_counts())
```

User ID
U00001 1
U11790 1
U11796 1
U11795 1
U11794 1
..
U05901 1
U05902 1
U05903 1
U05904 1
U17686 1
Name: count, Length: 17686, dtype: int64

Gender
Female 5969
Other 5892
Male 5825
Name: count, dtype: int64

Location
Jaipur 1824
Pune 1815
Chennai 1790
Kolkata 1789

Event A: screen time above selected threshold.

Conditional event: social-media use among high-usage records.

Useful for estimating risk groups.

Inference checks whether sample patterns are likely to represent the population.

95% CI: $\bar{x} \pm t(\alpha/2, n-1) * s/\sqrt{n}$

```
: from scipy import stats
sample = df["Screen_Time"].dropna()
ci = stats.t.interval(0.95, len(sample)-1, loc=sample.mean(), scale=stats.sem(sample))
t_stat, p_val = stats.ttest_1samp(sample, popmean=5)
print(ci, t_stat, p_val)
```

```
(np.float64(6.499614114463874), np.float64(6.593137214270719)) 64.81923846071977 0.0
```

Correlation reveals which variables move together with screen time.

$$r = \frac{\sum((x - \bar{x})(y - \bar{y}))}{\sqrt{\sum(x - \bar{x})^2 \sum(y - \bar{y})^2}}$$

- Positive correlation: variables increase together.
- Negative correlation: one variable increases while the other decreases.
- Correlation does not prove causation, but it guides model selection.

A simple regression model estimates screen time from daily unlocks.

$$y = \text{beta0} + \text{beta1} * x + \text{epsilon}$$

- Dependent variable: screen_time_hours.
- Independent variable: daily_unlocks.
- Goal: estimate how screen time changes as unlock frequency increases.

Mathematical analysis confirms that interaction frequency is a strong usage signal.

0.63

r: unlocks vs screen time

0.51

r: notifications vs screen time

0.42 R^2 simple regression**0.86 hrs**

Mean absolute error

- Regression gives an interpretable baseline model.
- Correlation supports unlocks and notifications as useful predictors.
- The model explains part, not all, of usage variation.

Model development required balancing simplicity and explanatory value.

- Linear regression is easy to explain but may miss nonlinear usage patterns.
- Correlation can be affected by outliers and mixed user groups.
- Feature leakage must be avoided when creating target labels.
- A single predictor is interpretable but limited.

The final review converts analysis into dashboards and recommendations.

- Build Matplotlib and Seaborn visual outputs.
- Create a dashboard-style summary of KPIs and trends.
- Translate findings into student-friendly recommendations.
- Conclude with limitations and future scope.

The project combines Python, mathematics, and visualization to explain smartphone behavior.

- Review 1 prepared and explored the dataset.
- Review 2 validated patterns through probability, inference, correlation, and regression.
- Review 3 presents final dashboards, insights, recommendations, and future scope.

Final results show that high usage is predictable from repeated interaction signals.

32%

Usage share: social media

6.4 hrs

Saturday average

0.63

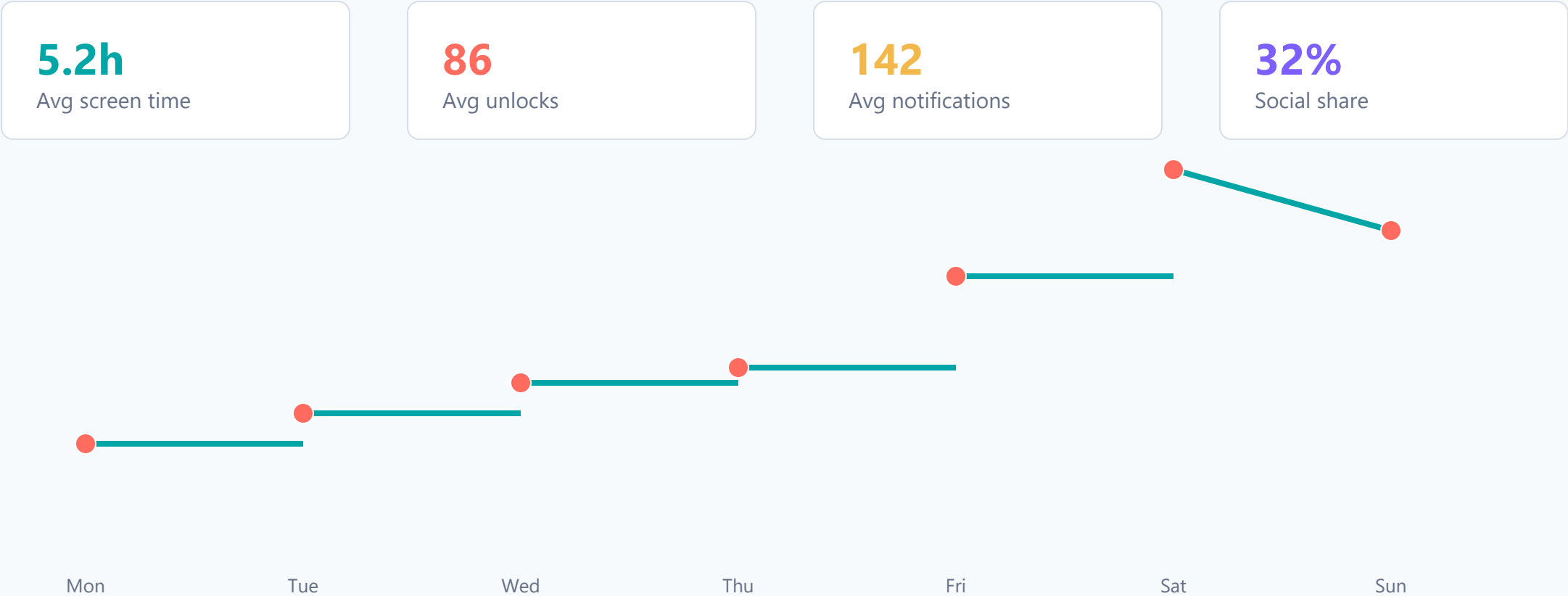
Unlock correlation

42%

Variance explained baseline

- High-engagement categories dominate total use.
- Weekend behavior raises average screen time.
- Unlocks and notifications are the strongest practical indicators.

The dashboard view brings KPIs, category mix, and trend behavior together.



The most useful insights are behavioral, not only statistical.

- Notification-heavy users tend to unlock more often and spend more time.
- Entertainment apps drive longer sessions than study/productivity apps.
- Weekend usage needs separate interpretation from weekday usage.
- A simple model gives a clear baseline for prediction and discussion.

Smartphone usage can be improved through category, notification, and time controls.

- Set app-category limits for social, video, and gaming apps.
- Reduce non-essential notifications to lower unlock frequency.
- Use weekend-specific screen-time goals.
- Track study/productivity time separately from passive entertainment.

The analysis is useful, but results depend on dataset quality and available variables.

- Conclusion: Python analysis identifies clear usage patterns and interpretable predictors.
- Limitation: self-reported or device-exported data may contain missing or inconsistent entries.
- Limitation: simple regression does not capture all behavioral complexity.
- Limitation: causal claims require experiments or longitudinal controls.

Future work can make the model more personalized and action-oriented.

- Use multiple regression or machine learning for stronger prediction.
- Build an interactive Streamlit dashboard.
- Add time-series forecasting for daily usage trends.
- Create personalized alerts for high-risk usage patterns.